

Checkpoint — Niveles 08 a 12

Nota de consolidación **en español**, a propósito. El resto del vault está en inglés; este repaso no. Lo que se revisa para fijar conviene leerlo en el idioma en que se piensa.

No repite las notas de nivel. Recoge solo lo que sigue siendo cierto cuando se olviden los comandos concretos.

Todos los ejemplos están probados y se pueden ejecutar. Los comandos que aún no hemos trabajado van comentados en la propia línea, y hay un glosario al final.

1. La idea que lo atraviesa todo

Un comando que se ejecuta sin error no es un comando que ha funcionado.

Cinco niveles, cinco veces el mismo susto:

Comando	Qué parecía	Qué hacía en realidad
<code>uniq -u data.txt</code>	filtrar líneas únicas	comparar solo líneas adyacentes → 981 falsos positivos
<code>base64 data.txt</code>	decodificar	codificar otra vez → base64 válido y más largo
<code>sort data.txt</code> dentro de una pipe	ordenar lo que llega	leer el fichero e ignorar la pipe
<code>xxd -r data.txt > data.txt</code>	transformar el fichero	vaciarlo antes de leerlo
<code>grep -a</code> sobre binario	encontrar la contraseña	encontrarla y devolverla ilegible

Ninguno falló. Ninguno avisó. Los cinco devolvieron algo plausible.

Un error es un regalo: te dice dónde mirar. Una salida silenciosamente equivocada no dice nada, y es la que acaba en un informe.

Por qué esto importa fuera de Bandit

Imagina que en un incidente cuentas intentos fallidos de login:

```
grep 'Failed password' auth.log | uniq -c
#                               └─ -c cuenta cuántas veces se repite cada línea
```

Sin `sort` delante, `uniq` solo agrupa lo adyacente. El comando **funciona**, devuelve una tabla con números, y esos números están mal. Informas de 12 intentos desde una IP cuando fueron 400.

Nadie te va a decir que te equivocaste. El comando no falló.

De ahí la única regla que hay que llevarse de esta semana: **comprobar la salida no es desconfianza, es parte del trabajo.**

2. Streams: saber quién lee de dónde

Todo programa tiene tres canales:

Canal	Número	Qué lleva
<code>stdin</code>	0	lo que el programa lee
<code>stdout</code>	1	la salida normal
<code>stderr</code>	2	los mensajes de error

Una pipe `|` conecta el `stdout` de un programa con el `stdin` del siguiente. Los errores viajan por un canal aparte y **no entran en la pipe** — por eso sigues viendo errores en pantalla aunque hayas redirigido la salida.

Lo que no es evidente y hay que memorizar: **cada comando decide de dónde lee, y no todos deciden igual.**

Grupo A — ignoran `stdin` si les das un fichero

`sort`, `grep`, `wc`, `head`, `tail`, `cut`, `uniq`

Pruébalo y velo con tus ojos:

```
printf 'zzz\nyyy\n' > fichero.txt
# printf escribe texto SIN añadir salto de línea automático (a diferencia de echo).
# Por eso los \n van escritos a mano: crean las dos líneas del fichero.

echo "AAA" | sort fichero.txt
```

Salida real:

```
yyy
zzz
```

`AAA` **no aparece por ningún lado.** `sort` abrió `fichero.txt` y tiró a la basura todo lo que venía por la pipe. Sin error, sin aviso.

Aplicado a tu fallo del nivel 9:

```
base64 -d data.txt | sort data.txt | grep "=" # ROTO
base64 -d data.txt | sort | grep "="          # correcto
```

Grupo B — solo leen de `stdin`, no aceptan fichero

`tr` es el caso claro:

```
tr 'A-Za-z' 'N-ZA-Mn-za-m' data.txt          # error: no existe esa forma
cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'     # correcto
tr 'A-Za-z' 'N-ZA-Mn-za-m' < data.txt         # correcto y con un proceso menos
```

Los dos argumentos de `tr` son los **conjuntos de caracteres**, nunca los datos. Por eso `tr 'Gur cnffjbeq...' daba missing operand: le pasaste el contenido del fichero como si fuera un conjunto.`

< fichero frente a `cat fichero |` : hacen lo mismo. El primero le dice al shell "conecta este fichero a la entrada del programa"; el segundo lanza un proceso `cat` extra solo para volcar el contenido. Con ficheros grandes se nota. `cat file | grep x` funciona; `grep x file` es mejor.

Cómo saberlo sin memorizar

```
sort --help | head -3
# Usage: sort [OPTION]... [FILE]...      ← acepta ficheros

tr --help | head -3
# Usage: tr [OPTION]... SET1 [SET2]      ← no hay [FILE]: solo stdin
```

Si la línea `Usage:` incluye `[FILE]` , acepta ficheros. Si no, es stdin-only.

Regla práctica: en una pipeline, solo el primer comando lleva nombre de fichero.

Cómo se ejecutan realmente las pipes

No hay turnos ni prioridades: **todos los comandos arrancan simultáneamente**. Lo que hay es dependencia de datos — el segundo se queda bloqueado hasta que el primero produce algo.

Tres consecuencias prácticas:

`head` mata lo que tiene encima. Cuando ya tiene sus líneas, cierra la entrada y el comando de la izquierda recibe una señal `SIGPIPE` — el aviso del sistema de que "nadie está leyendo tu salida"— y muere. Por eso `head -5 log_de_40GB` responde al instante: no lee 40 GB y descarta, **corta la lectura de verdad**.

`sort` es el cuello de botella. No puede emitir la primera línea hasta haber leído la última, porque hasta entonces no sabe cuál va primera. `grep` en cambio va emitiendo según lee.

El código de salida es el del último comando. Todo programa devuelve al terminar un número: 0 = éxito, cualquier otro = error. En una pipeline solo cuenta el del último:

```
false | true    # `false` es un comando que solo sirve para fallar,
                 # `true` para tener éxito. El conjunto devuelve 0 (éxito)
                 # aunque la primera etapa haya fallado.
```

Si quieres que la pipeline falle cuando falle cualquier etapa, se activa una opción del shell:

```
set -o pipefail  # a partir de aquí, la pipeline devuelve error si CUALQUIER etapa falla
```

3. Las redirecciones ocurren antes que el comando

Esta es la que cuesta dinero. Pruébala en un fichero de mentira:

```
printf 'linea1\nlinea2\nlinea3\n' > datos.txt

wc -c < datos.txt      # → 21
```

```
# └─ -c cuenta BYTES (caracteres). Ya conoces -l, que cuenta líneas.

sort datos.txt > datos.txt

wc -c < datos.txt          # → 0
```

El fichero desaparece. Y `sort` ni siquiera se quejó.

El orden real de lo que hace bash

1. Lee la línea completa y **encuentra** `> datos.txt`
2. Abre ese fichero y **lo trunca a 0 bytes** — aquí ya se perdió todo
3. Ahora sí, lanza `sort`
4. `sort` abre `datos.txt` ... y lo encuentra vacío

La destrucción ocurre **antes de que el comando exista**. No es un fallo de `sort`; es cómo funciona el shell, y le pasa igual a `sed`, `awk`, `grep`, `tr` y `xxd`.

En `cmd origen > destino`, **origen y destino nunca pueden ser el mismo fichero**. Ni directamente, ni después de un `mv` que te haya hecho perder la pista de dónde están los datos.

La forma correcta

```
sort datos.txt > tmp.txt && mv tmp.txt datos.txt
# └─ && = "y si lo anterior tuvo éxito, entonces...".
#      Si sort falla, el mv NO se ejecuta y conservas el
original.
#      Ver [[Linux - Logical Operators and Test Conditions]]
```

Casos reales del mismo patrón:

```
# Quitar los retornos de carro que Windows mete al final de cada línea
tr -d '\r' < script.sh > tmp.sh && mv tmp.sh script.sh
# └─ -d BORRA los caracteres del conjunto en vez de sustituirlos.
#      '\r' es el byte de retorno de carro (CR) que usa Windows.

# Eliminar de un log todas las líneas que contengan DEBUG
grep -v 'DEBUG' app.log > tmp.log && mv tmp.log app.log
# └─ -v INVIERTE el filtro: muestra las líneas que NO coinciden.
```

La excepción: `sed -i`

`sed` es un editor de texto que trabaja sin abrir nada: recibe líneas, les aplica una regla y las devuelve. Su uso más común es sustituir texto:

```
sed 's/viejo/nuevo/g' fichero.txt
# | | | |
# | | | └─ g = "global": todas las apariciones de cada línea,
# | | |       no solo la primera
# | | └─ texto de reemplazo
```

```
# | _____ texto a buscar
# | _____ s = "substitute" (sustituir)
```

Eso imprime el resultado en pantalla sin tocar el fichero. Para modificarlo de verdad existe `-i` (*in place*), que hace el fichero temporal por dentro y es seguro:

```
sed -i 's/viejo/nuevo/g' fichero.txt      # modifica el fichero directamente
sed -i.bak 's/viejo/nuevo/g' fichero.txt  # igual, pero deja copia en
fichero.txt.bak
```

`-i` es la única forma segura de "editar sobre el mismo fichero", porque el temporal lo gestiona la herramienta y no tú.

4. El nombre miente, los bytes no

En Linux la extensión es **una convención humana sin ningún valor**. El sistema no la usa para nada. Puedes llamar `foto.jpg` a un ejecutable y se ejecutará igual.

Lo que sí identifica un fichero es su **magic number**: una firma de bytes al principio.

Bytes	ASCII	Formato
1f 8b		gzip
42 5a 68	BZh	bzip2
50 4b 03 04	PK	zip, docx, xlsx, pptx, jar, apk
4d 5a	MZ	ejecutable Windows
7f 45 4c 46	.ELF	ejecutable Linux
25 50 44 46	%PDF	PDF
89 50 4e 47	.PNG	PNG

Demostración

```
printf 'hola mundo\n' > texto.txt

gzip -c texto.txt > comprimido.gz
#     | -c escribe a stdout en vez de reemplazar el fichero.
#     | Sin -c, gzip borraría texto.txt y dejaría texto.txt.gz.

mv comprimido.gz mentira.txt      # le ponemos un nombre falso

file -b mentira.txt
#     | -b = "brief": imprime solo el tipo, sin repetir el nombre del fichero
#     | delante
```

Salida real:

```
gzip compressed data, was "texto.txt", last modified: ..., original size modulo 2^32
11
```

El nombre dice `.txt` . file dice `gzip`. Y además recupera el nombre original, porque `gzip` lo guarda dentro de su cabecera.

Al revés también funciona:

```
printf 'esto es texto plano\n' > falso.gz
gunzip falso.gz
# → gzip: falso.gz: not in gzip format
```

Le pusiste `.gz` y no coló. `gunzip` leyó los bytes.

El matiz que confunde

`gunzip` **sí exige** la extensión `.gz` para ponerse a trabajar. Pero eso es una **comprobación de seguridad suya** —para saber cómo nombrar la salida y no destrozarse un fichero que no era lo que creías—, no la forma en que averigua el formato.

Dos cosas distintas que parecen la misma:

	Para qué sirve
La extensión	que <code>gunzip</code> acepte ejecutarse y sepa cómo llamar al resultado
El <code>magic number</code>	saber qué es el fichero de verdad

Por eso `gunzip -c` funciona sin extensión: al escribir a `stdout` no tiene que inventar ningún nombre, y la comprobación sobra.

Cómo mirar los bytes sin riesgo

```
file fichero # la respuesta directa

head -c 16 fichero | xxd # los 16 primeros BYTES en hexadecimal
#   └─ -c cuenta bytes en lugar de líneas. Imprescindible en binarios,
#       donde los saltos de línea aparecen en posiciones aleatorias.
```

Nunca hagas `cat` a un binario: los bytes de control pueden dejarte el terminal inutilizable. Si pasa, se arregla con `reset` .

Aplicación real

Que un `.docx` empiece por `PK` significa que **es un zip**. Puedes inspeccionar un documento sospechoso sin abrir Office:

```
unzip -l factura.docx
#   └─ -l = "list": muestra qué contiene SIN extraer nada

unzip -p factura.docx word/document.xml | head -c 500
```

```
# └─ -p = "pipe": vuelca el contenido de UN fichero interno a stdout,  
# sin escribir nada en el disco. Aquí se cortan los primeros 500 bytes.
```

Si aparecen macros o referencias externas, tienes la respuesta sin haber ejecutado nada.

5. Codificar no es cifrar

Tres operaciones que se confunden constantemente:

	Para qué	¿Clave?	¿Reversible?
Codificar	que el dato sobreviva a un canal	No	Sí, por cualquiera
Cifrar	que nadie no autorizado lo lea	Sí	Solo con la clave
Hashear	verificar integridad	No	No, por diseño

base64

Convierte cualquier byte a 64 caracteres imprimibles. Cada 3 bytes se vuelven 4 → **crece un 33%**.

```
echo -n "secreto" | base64  
# └─ -n evita que echo añada un salto de línea al final (ver más abajo)  
# → c2VjcmV0bw==  
  
echo -n "c2VjcmV0bw==" | base64 -d  
# → secreto
```

Sin clave. Sin secreto. Cualquiera lo revierte.

El fallo del nivel 10, reproducido:

```
echo -n "secreto" | base64 | base64  
# → YzJWamNtVjBidz09Cg== ← lo codificó DOS veces, sin error
```

Y una trampa que te va a morder algún día — `echo` añade un salto de línea que se codifica junto con el dato:

```
echo "secreto" | base64 # → c2VjcmV0bwo= ← la "o=" del final es el \n  
echo -n "secreto" | base64 # → c2VjcmV0bw== ← correcto
```

Codificar una contraseña con ese `\n` colado produce un valor que falla la autenticación por motivos imposibles de ver a simple vista.

ROT13

Desplaza cada letra 13 posiciones. Como 13 es la mitad de 26, **es su propio inverso**:

```
echo "Hola Mundo" | tr 'A-Za-z' 'N-ZA-Mn-za-m'  
# → Ubyn Zhaqb
```

```
echo "Hola Mundo" | tr 'A-Za-z' 'N-ZA-Mn-za-m' | tr 'A-Za-z' 'N-ZA-Mn-za-m'
# → Hola Mundo
```

El mismo comando codifica y decodifica. No hay `-d` que olvidar.

Por qué esto importa de verdad

Autenticación HTTP Basic es literalmente base64:

```
echo -n "admin:P4ssw0rd" | base64
# → YWRtaW46UDRzc3cwcmQ=
```

Esa cadena viaja en la cabecera `Authorization: Basic`. Sin HTTPS, cualquiera que capture el tráfico la revierte en un segundo. **Por eso Basic Auth sin TLS es un hallazgo de auditoría.**

Los tokens JWT llevan la carga útil en base64. La firma impide *modificarlos*, no *leerlos*. Meter datos personales en un JWT es un fallo de diseño recurrente.

Contraseñas en ficheros de configuración etiquetadas como `encryptedPassword` que en realidad son base64. Buscarlas y decodificarlas es rutina en cualquier pentest.

Clasificar antes de actuar

Pista	Probablemente es
<code>A-Za-z0-9+ /</code> con <code>=</code> al final, longitud múltiplo de 4	base64
Solo <code>0-9a-f</code> , longitud par	hexadecimal
<code>%20</code> , <code>%2F</code>	codificación URL
Se lee raro pero con estructura (<code>Gur</code> = <code>The</code>)	ROT13
Exactamente 32 / 40 / 64 caracteres hex	MD5 / SHA-1 / SHA-256 — no se revierte
Empieza por <code>\$2b\$</code> o <code>\$6\$</code>	hash de contraseña (bcrypt / SHA-512)
Alta entropía, sin estructura ni relleno	cifrado de verdad — busca la clave

Las dos últimas filas son las que ahorran tiempo: reconocer un **hash** significa dejar de intentar decodificarlo, y reconocer **cifrado real** significa que el problema es dónde está la clave, no qué herramienta usar.

6. El método, que vale más que los comandos

Los comandos se olvidan y se buscan. Esto no.

Mirar antes de actuar

En el nivel 8, antes de resolver nada, ejecutaste tres comandos que no resolvían nada:

```
wc -l data.txt           # → 1001    ¿cuántas líneas hay?
uniq -u data.txt | wc -l # → 981     ¿qué pasa sin ordenar?
sort data.txt | uniq -c | head
```



```
# → 10 0LTDNpAmqqfuE0FlE0ksGF6c0Kraspzs
# 10 1cKKjk7M0Pl2cPubYgc9W4307bYC0ohF
```

Con eso reconstruiste la estructura: **100 líneas × 10 repeticiones + 1 única = 1001**. Los tres comandos eran innecesarios para la respuesta e imprescindibles para que la respuesta fuera **fiable**.

Repertorio de reconocimiento:

```
head -3 fichero      # ¿qué formato tiene esto?
file fichero          # ¿qué es realmente?
wc -l fichero         # ¿de cuánto hablamos?
comando | head -30    # ¿qué produce esta herramienta nueva?
```

Construir por etapas

Este ejemplo usa `awk`, que aún no hemos trabajado. Lo único que necesitas saber por ahora: `awk` **parte cada línea por espacios y te deja quedarte con la columna que quieras**. `$1` es la primera columna, `$2` la segunda, etc. En un log de acceso web, la primera columna es la IP del cliente.

```
awk '{print $1}' access.log | head -5
#   └─ imprime solo la columna 1 de cada línea. ¿Es realmente la IP?

awk '{print $1}' access.log | sort | head -5
#                               └─ ¿ordena como espero?

awk '{print $1}' access.log | sort | uniq -c | head -5
#                               └─ ¿agrupa y cuenta bien?

awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -20
#                               └─ -r invierte el orden,
#                               -n compara como NÚMEROS
#                               → las IPs más repetidas arriba
```

Ese último comando es **el más usado en análisis de logs**: te da el ranking de quién ha hecho más peticiones. Una IP con 40.000 frente a una media de 30 no es sutil.

Una pipe de cuatro etapas escrita de golpe y depurada a ciegas no te dice **qué etapa** falló.

Listar antes de destruir

```
find . -name "*.tmp" | head -20    # mira qué coincide
find . -name "*.tmp" -delete       # solo entonces
```

En PowerShell el equivalente es `-WhatIf`, que ya usaste al limpiar los `.*`. Distinta herramienta, mismo reflejo.

Documentación local antes que buscador

```
base64 --help    # instantáneo, de tu versión exacta, sin red
man sort
```

El detour del `-m32` salió de leer un banner en vez de leer `base64 --help` . Un segundo habría bastado.

Comandos consolidados

Comando	Para qué	Trampa principal
<code>sort</code>	ordenar	sin <code>-n</code> compara como texto: <code>"10" < "9"</code>
<code>uniq</code>	duplicados	solo compara líneas adyacentes → exige <code>sort</code> delante
<code>wc -l</code>	contar	verificar cada etapa de una pipeline
<code>head / tail</code>	muestrear	<code>tail -F</code> (mayúscula) para logs que rotan
<code>strings</code>	texto dentro de binarios	por defecto solo runs de 4+ caracteres
<code>grep</code>	filtrar líneas	filtra contenido , no rutas ni nombres
<code>base64 -d</code>	decodificar	sin <code>-d</code> codifica
<code>tr</code>	sustituir caracteres	solo lee stdin, jamás fichero
<code>file</code>	identificar formato	lee bytes, ignora el nombre
<code>xxd -r</code>	revertir hexdump	nunca al mismo fichero
<code>find</code>	buscar	sin <code>-name</code> , un argumento suelto es una ruta de inicio

Glosario — lo que aparece aquí y aún no hemos trabajado

Ninguno hace falta para el nivel 13. Están aquí para que los ejemplos se lean sin descifrarlos.

Comandos

Comando	Qué hace	Cuándo lo verás en serio
<code>printf 'a\nb\n'</code>	escribe texto sin añadir salto de línea automático; los <code>\n</code> van a mano. Más predecible que <code>echo</code>	scripting
<code>awk '{print \$1}'</code>	parte cada línea por espacios e imprime la columna indicada (<code>\$1</code> , <code>\$2</code> ...)	análisis de logs
<code>sed 's/x/y/g'</code>	sustituye texto línea a línea. <code>s</code> =sustituir, <code>g</code> =todas las apariciones	edición automatizada
<code>gzip -c</code>	comprime escribiendo a stdout en vez de reemplazar el fichero	ya lo usaste en el nivel 12
<code>unzip -l / -p</code>	listar contenido / volcar un fichero interno sin extraer	análisis de documentos
<code>false / true</code>	comandos que solo sirven para fallar o tener éxito. Para probar lógica	scripting
<code>reset</code>	reinicia el terminal cuando se corrompe	cuando hagas <code>cat</code> a un binario

Opciones que se repiten

Opción	Significado	Aparece en
<code>-c</code>	según el comando: contar (<code>uniq -c</code>), bytes (<code>wc -c</code> , <code>head -c</code>), o stdout (<code>gzip -c</code> , <code>gunzip -c</code>)	muchos
<code>-v</code>	invertir el filtro: muestra lo que NO coincide	<code>grep -v</code>

Opción	Significado	Aparece en
<code>-d</code>	borrar o decodificar	<code>tr -d</code> , <code>base64 -d</code>
<code>-n</code>	numérico (<code>sort -n</code>) o sin salto de línea (<code>echo -n</code>)	<code>sort</code> , <code>echo</code>
<code>-r</code>	invertir el orden (<code>sort -r</code>) o recursivo (<code>cp -r</code>) o revertir (<code>xxd -r</code>)	varios
<code>-b</code>	brief: salida escueta	<code>file -b</code>

Que la misma letra signifique cosas distintas según el comando **no es un despiste tuyo**. No hay estándar: cada herramienta eligió sus letras por su cuenta hace décadas. Por eso `--help` no es opcional.

Sintaxis del shell

Símbolo	Significado
<code> </code>	pipe: stdout de la izquierda → stdin de la derecha
<code>></code>	redirige stdout a un fichero (lo crea o vacía)
<code>>></code>	redirige stdout añadiendo al final
<code><</code>	alimenta stdin desde un fichero
<code>2></code>	redirige solo stderr
<code>2>/dev/null</code>	descarta los errores
<code>&&</code>	ejecuta lo siguiente solo si lo anterior tuvo éxito
<code>\\ </code>	ejecuta lo siguiente solo si lo anterior falló
<code>\$(cmd)</code>	ejecuta <code>cmd</code> y sustituye por su salida

`&&` y `||` los tienes en [Linux - Logical Operators and Test Conditions](#); el resto en [Linux - Piping and Redirection](#).

Autoevaluación

Si puedes responder esto sin mirar, el checkpoint está superado:

- ¿Por qué `cat a.txt | grep x b.txt` no busca en `a.txt` ?
- ¿Qué queda en `f.txt` tras `sort f.txt > f.txt` , y en qué momento exacto se pierde?
- ¿Cómo aplicas ROT13 al contenido de un fichero, y por qué no vale pasarle el nombre a `tr` ?
- Un fichero se llama `backup.gz` pero `file` dice `POSIX tar archive` . ¿Quién tiene razón y por qué?
- ¿Qué diferencia hay entre `echo "x" | base64` y `echo -n "x" | base64` ?
- ¿Por qué `uniq -c` sin `sort` delante da números incorrectos?

Dónde estás

Cerrado: Bandit 8→12. Procesamiento de texto, codificaciones, formatos binarios y compresión anidada.

Lo que viene: el 13 cambia de tema — autenticación por clave SSH. Se acaba el tratamiento de ficheros y empiezan credenciales y red, que enlaza con el Dominio 2 de Kasiu.

Lo que no hay que arrastrar: dos veces se usó `find` como reflejo de apertura para localizar un fichero que estaba en el directorio actual. La corrección es `pwd` y `ls -la` primero. Reconocimiento barato antes que herramienta cara.

Notas relacionadas

- [Linux - Piping and Redirection](#) — streams, pipes y el caso `tr`
- [Linux - Sorting and Deduplication](#) — `sort`, `uniq`, `wc` y análisis de logs
- [Linux - Extracting Strings from Binaries](#) — `strings` y análisis estático
- [Linux - Sampling and Inspecting Output](#) — `head`, `tail`, `less`
- [Linux - Encoding vs Encryption](#) — base64, ROT13, cifrado y hash
- [Linux - Nested Archives and Compression Layers](#) — magic numbers y compresión
- [Linux - File Type Detection](#) — `file` y el concepto de fondo
- [Linux - Logical Operators and Test Conditions](#) — `&&`, `||` y condicionales